Agile Methodology

Quick Revision Guide

Understand the Agile way of building software — explained simply with real-world analogies

November 2026

1. What is Agile?

Agile is a way of building software where you **don't try to build everything at once**. Instead, you break the project into small pieces, build one piece at a time, deliver it, get feedback, and then build the next piece. It's an iterative and incremental approach.

Core Idea: Build small, deliver fast, learn, repeat.

The Restaurant Analogy

Waterfall (The Old Way)

Imagine building a restaurant. You spend **6 months planning everything** — kitchen, dining hall, bar, rooftop, parking. Then you build it all over 2 years. On opening day, you discover... customers wanted a cafe, not a bar. **Months of work wasted** on something nobody wanted, and it's too late to easily change it.

Agile (The Smart Way)

You open with **just the kitchen and 5 tables**. Customers start eating on Day 1. They say "we need more seating" — so you add tables next week. Then they say "outdoor seating would be great" — so you build that next. You're **delivering value from week one** and building based on **real feedback**.

The Three Pillars of Agile

1. Break It Down	2. Deliver in Cycles	3. Get Feedback & Adapt
Big project → small manageable pieces (Epic → Feature → Story → Task)	Build and ship one piece every 2 weeks (called a Sprint)	After each sprint, learn what users want and adjust the plan

2. The Agile Hierarchy

Agile organizes work in a clear top-down structure. Each level breaks the work into smaller, more specific pieces.

EPIC — The Big Goal
Too large to build at once. E.g., "Customer Service"

FEATURE — Major Sections
Meaningful chunks of the epic. E.g., "Your Orders", "Returns"

USER STORY — What Users Do
Actions from a user's perspective. E.g., "Track Packages"

TASK — Developer Work
Actual work items. E.g., "Build React UI — 5 hrs"

Notice how each level gets narrower — that shows how one Epic contains many Features, each Feature has many Stories, and each Story has many Tasks.

3. Real Example: Customer Service Epic

Epic: Customer Service

Features:

Your Orders	Returns & Refunds	Manage Address	Manage Prime

User Stories (for "Your Orders"):

Track Packages	Edit or Cancel Orders

Tasks (for "Track Packages"):

Task	Hours
Build the React UI	5 hrs
Build the FastAPI Layer	6 hrs
Build the DB Schema	8 hrs
Write Unit Test Cases	4 hrs
Configure CI/CD Pipeline	2 hrs

TOTAL	**25 hrs**

4. How Sprints Work

A **Sprint** is a short, fixed time period (usually 2 weeks) where the team picks a set of user stories, builds them, and delivers a working product. Here's how the Customer Service epic might be built across sprints:

Sprint	What Gets Built	Result
Sprint 1–2	Track Packages	Users can already track orders!
Sprint 3–4	Edit / Cancel Orders	"Your Orders" feature complete
Sprint 5–6	Return & Exchange Items	Returns feature begins
Sprint 7–8	Print Labels + Manage Address	More features go live
Sprint 9–10	Manage Prime	Full epic delivered

After every sprint, the team gets feedback and can adjust priorities for the next sprint.

5. Agile vs Waterfall — Side by Side

	Waterfall	Agile
Approach	Build everything, then deliver	Build & deliver piece by piece
Planning	Plan everything upfront	Plan as you go each sprint
Feedback	Only at the end	After every 2-week sprint
Changes	Expensive & difficult	Easy — expected & welcomed
Delivery	One big release at the end	Working software every sprint
Risk	High — discover problems late	Low — catch issues early

6. Key Takeaways

Epic

The big goal — too large for one team to tackle at once.

Feature

Major sections of the epic that users care about.

User Story

What a real user wants to do — written from their perspective.

Task

Actual developer work — with clear hour estimates.

Sprint

A 2-week cycle where the team builds, delivers, and gets feedback.

Remember: Agile = Break it down + Deliver in small cycles + Adapt based on feedback